fundamental idea of DDD is the bounded context. To map a part of the business domain—the subdomain—it makes sense to draw a boundary that allows you to talk and write about it clearly. The language you use to talk about the entities or events within a subdomain should not be used outside it.

Even if the concept of the customer can be considered as one and the same entity in a company, it is almost certain that different departments will only deal with certain aspects of the customer that are relevant to that particular department. Therefore, you should not treat it as the same entity. Different facets of the customer form different entities. Abstracting these different entities into one customer reduces precision, ultimately reducing the concept's usability in its different contexts. Abstracting several things into one entity forces each department to deal with attributes that are irrelevant to its particular business. The right level of abstraction should be found by business experts who have developed their concepts from experience. A *bounded context* is a subset of the business domain with the specific language used to refer to its parts.

There are many great ideas in DDD, but this is not the place to introduce them. A lot has been written, starting with the books *Domain Driven Design* by Eric Evans (Addison-Wesley Professional, 2003, http://mng.bz/VpJx) and *Implementing DDD* by Vaughn Vernon (Addison-Wesley, 2013, http://mng.bz/AlJQ). I encourage you to take a look if possible. Integration through SOA and MOM; isolation in time, space, failure, state, and deployment; microservices; CQRS; event-driven architectures; and DDD are ideas widely used in Akka. They will help you better understand this book and distributed systems in general.